

FRAUDLENS ENGINE

KELOMPOK 6



Naufal Rahman 2406413142



Ahmad Malik Prasetyo 2406416270



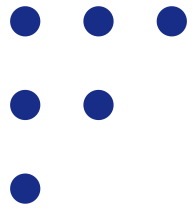
Eugenia Huwaida Imtinan 2406421384



Haikal Gifari Inzaghi 2406432261



Nadia Izzati 2406487033



Presentation Outline

- The Real-World Case Study
- Creative Application: Problem, motivation, and system architecture.
- Data Modeling
- Lessons Learned
- Individual Contributions



THE REAL-WORLD CASE STUDY

BNP Paribas Personal Finance, anak perusahaan dari raksasa perbankan Eropa BNP Paribas Group, **memproses lebih dari 800.000 pengajuan kredit setiap tahunnya. Sebelumnya, mereka menggunakan database relasional (SQL) yang kaku dan sangat lambat** saat harus **mengolah riwayat data yang saling tumpang tindih** (complex JOINS). **Untuk mengatasi taktik penipuan modern, mereka beralih menggunakan Neo4j.** Dengan Neo4j, sistem BNP Paribas kini mampu **menganalisis hubungan graf secara real-time** untuk **mendeteksi profil manipulatif**, misalnya menemukan **ribuan akun berbeda** yang ternyata menggunakan **device yang sama.** **Transisi** ke graph database **ini sukses memangkas waktu query menjadi kurang dari 2 detik** dan secara signifikan berhasil **menurunkan tingkat lolosnya penipuan hingga 20%** tanpa harus memblokir nasabah yang sah.



Sumber:

<https://neo4j.com/customer-stories/bnp-paribas-personal-finance/>

LOCAL CASE CONTEXT

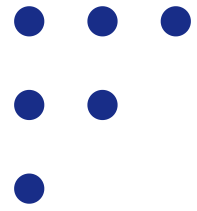
Sindikrat di Jakbar Kumpulkan 4.324 Rekening Judol Selama 30 Bulan

Dalam pengungkapan oleh Polres Metro Jakarta Barat, **sebuah sindikrat** terbukti **telah mengeksploitasi dan mengumpulkan 4.324 rekening penampung judi online selama 30 bulan**. Sindikrat ini beroperasi dengan taktik **synthetic identity berskala besar**, yaitu **mereka** menyewa identitas (KTP) warga asli, **membuka ribuan akun bank**, lalu **mengendalikan seluruh akses mobile banking dan nomor telepon OTP dari perangkat yang tersentralisasi**. Skala operasi yang **melibatkan ribuan entitas data yang saling tumpang tindih** ini yang **membuat sistem keamanan bank kewalahan** sehingga **membutuhkan pendekatan analisis relasi berbasis Graph Database** untuk membongkar jaringannya secara utuh.



Sumber:

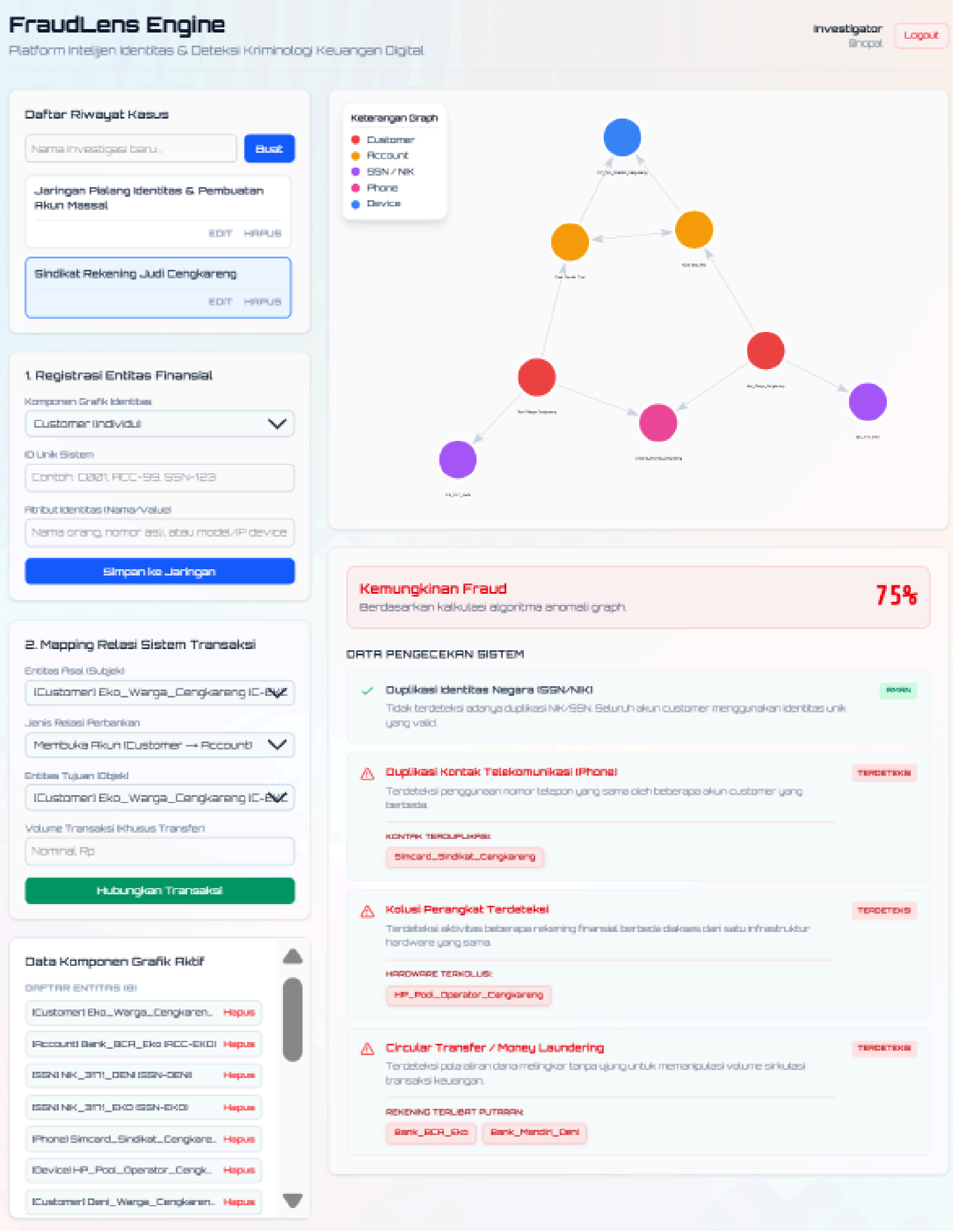
<https://megapolitan.antaranews.com/berita/319541/sindikrat-di-jakbar-kumpulkan-4324-rekening-judol-selama-30-bulan-sejak-mei-2022>



THE CORE PROBLEM

Dari kasus tersebut, **akar masalahnya** terletak pada **keterbatasan arsitektur database relasional** (SQL) yang digunakan oleh mayoritas institusi keuangan. **Data nasabah tersimpan secara terisolasi**. Karena pelaku menyewa KTP asli, **sistem bank melihat profil** tersebut sebagai **entitas yang valid dan sah**. Sistem **konvensional sangat lambat** dan **kesulitan membongkar hubungan berlapis**, seperti **melacak ribuan akun berbeda yang ternyata login dari satu perangkat yang sama**, karena membutuhkan query JOIN yang sangat berat. Akibatnya, **pola manipulatif ini gagal dicegah saat proses pembukaan rekening**.



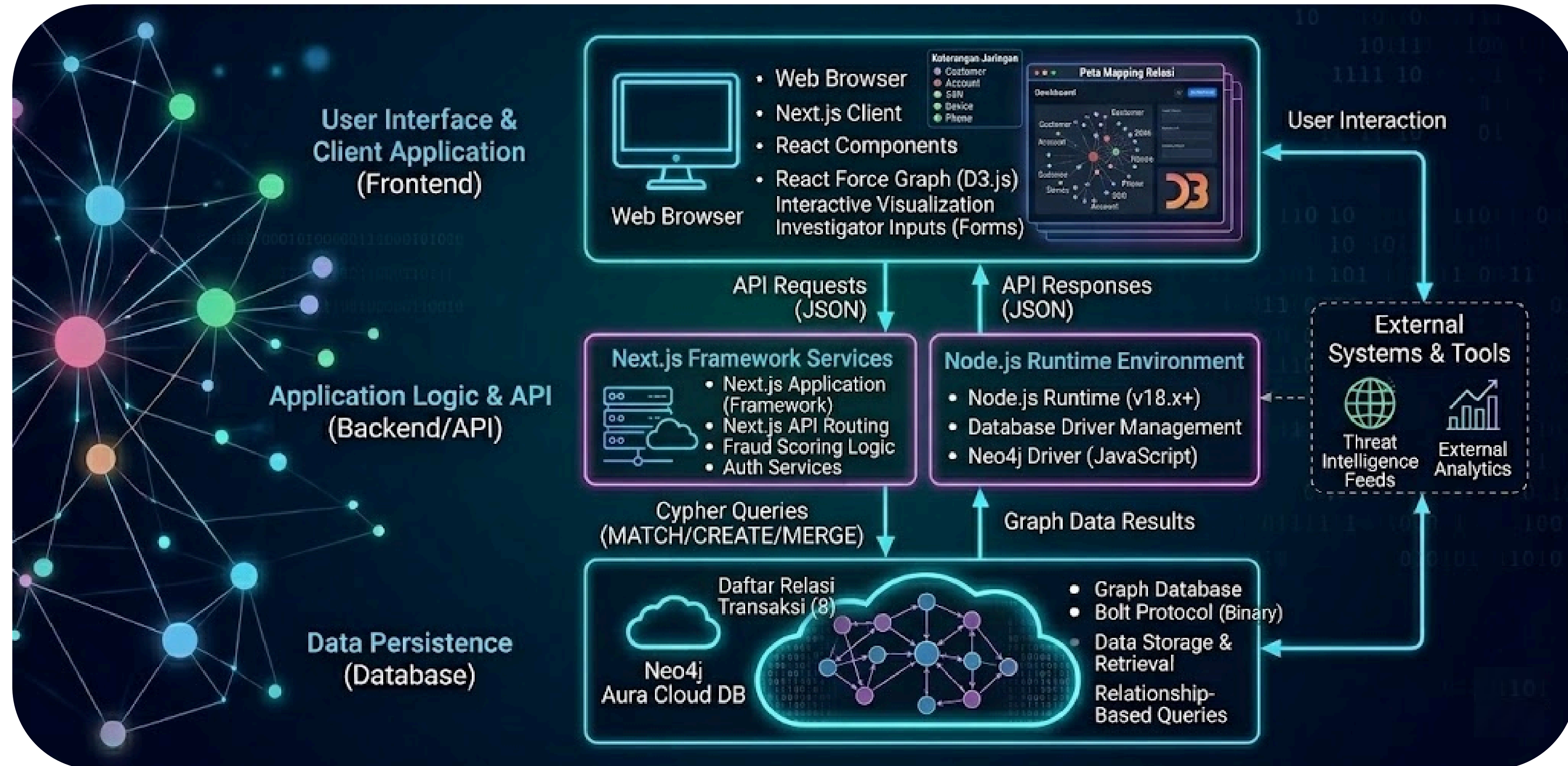


OUR MOTIVATION

Kelemahan sistem relasional konvensional memotivasi kami untuk mengembangkan **FraudLens Engine**, platform intelijen berbasis **Graph Database (Neo4j)**. Kami ingin **mengubah cara kerja investigator**, dari yang **awalnya mengolah ribuan baris tabel data yang membingungkan**, menjadi **pemetaan visual interaktif yang instan**.

FraudLens Engine secara otomatis **merangkai kepingan data yang terisolasi menjadi satu jaringan utuh**, mendeteksi **anomali transaksi**, dan **menghitung kemungkinan terjadinya fraud**. Dengan visibilitas ini, **sindikatan penipuan dapat langsung diputus sebelum kerugian finansial terjadi**.

SYSTEM ARCHITECTURE

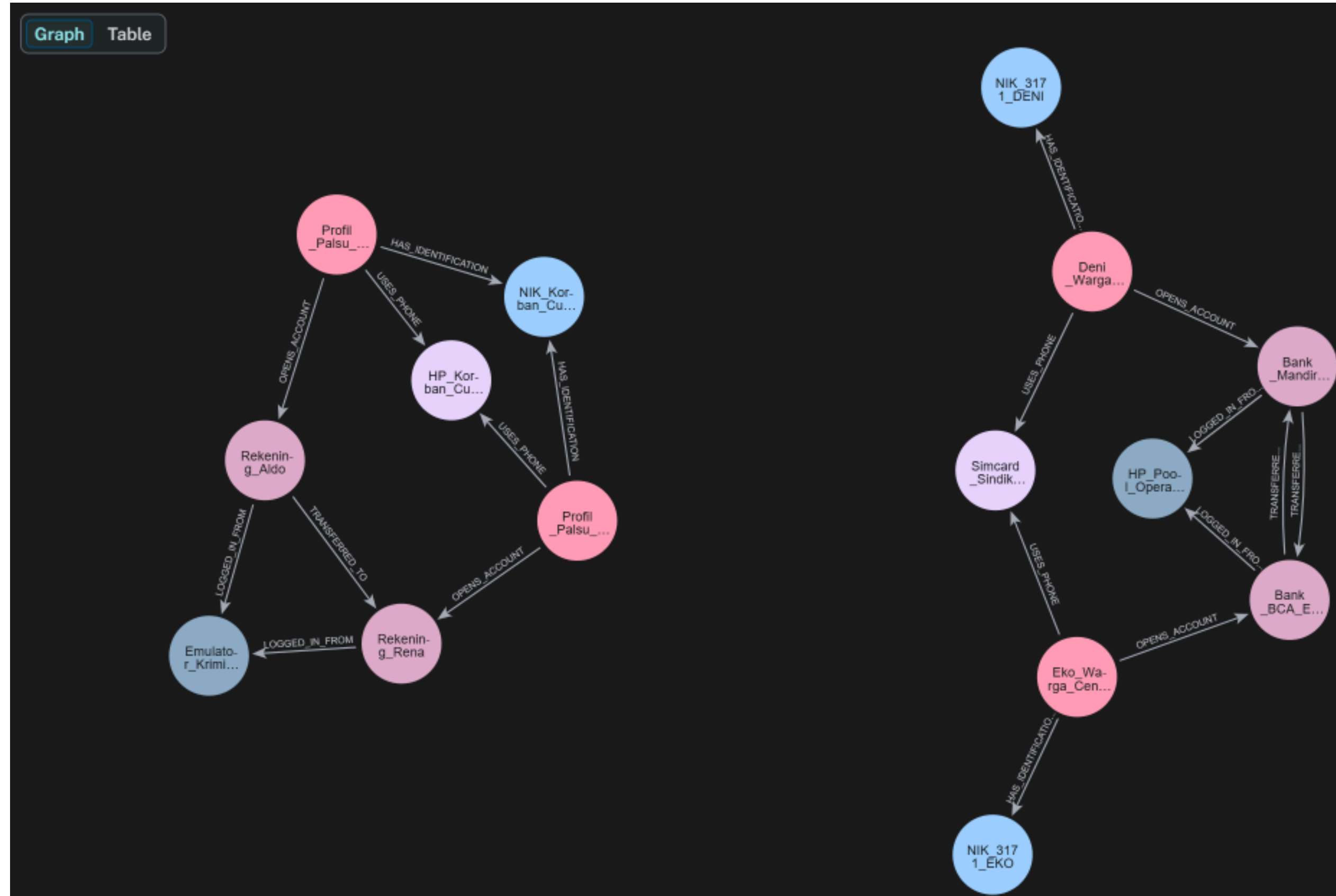


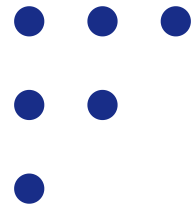
DATA MODELLING

Kami menstrukturkan data menjadi dua elemen utama: **Nodes** (simpul lingkaran) dan **Relationships** (garis panah).

Entitas fisik dan digital seperti Profil Pelaku (**Customer**), **NIK** (SSN), Nomor HP (**Phone**), dan Perangkat (**Device**) **direpresentasikan sebagai Nodes**. **Interaksi dan rekam jejak di antara mereka** ditandai oleh **Relationships** berarah, seperti **USES_PHONE**, **LOGGED_IN_FROM**, dan **TRANSFERRED_TO**.

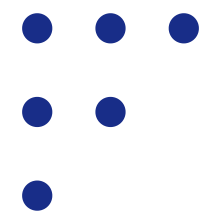
Pemodelan data seperti ini **sangat ideal** untuk **mendeteksi kejahatan finansial bersindik**. Kita bisa secara langsung melihat **pola anomali**, misalnya pada **kluster sebelah kanan**, terlihat bagaimana **dua akun rekening yang berbeda secara aktif dikendalikan dari satu titik Device yang sama dan saling mentransfer dana secara sirkular**. Pendekatan relasional ini **memungkinkan algoritma kami menelusuri jejak penipuan berlapis secara real-time**, sesuatu yang akan sangat lambat dan membebani server jika dieksekusi menggunakan query JOIN pada arsitektur SQL konvensional





LESSONS LEARNED

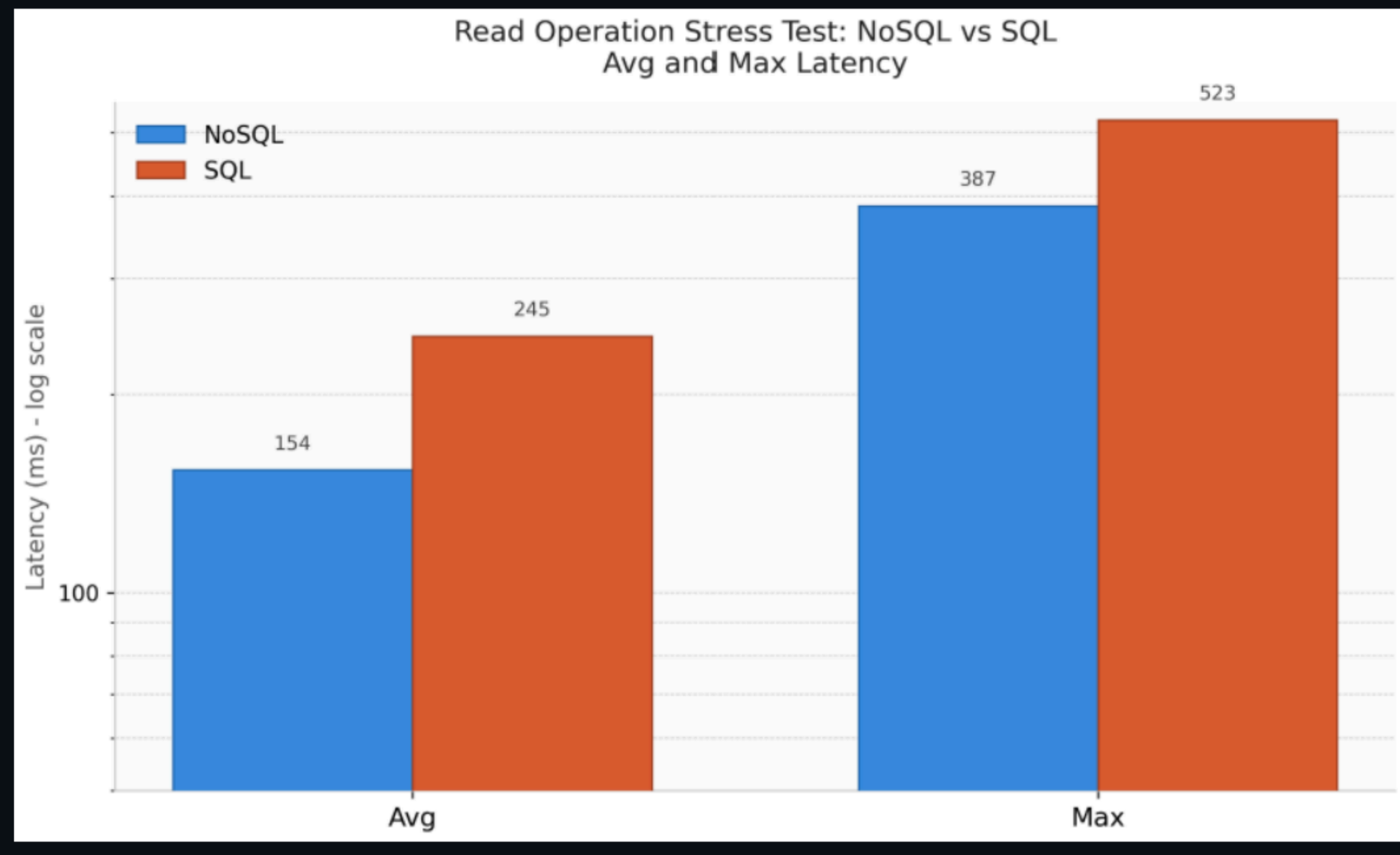
Membuat FraudLens Engine menuntut kami melakukan **trade-off waktu dan kenyamanan demi hasil yang maksimal**. Di sisi **backend**, meskipun kami belum terbiasa menggunakan Neo4j dan terbiasa menggunakan SQL konvensional, tapi dengan kami mempelajari cara menggunakannya, didapatkan performa pelacakan sindikat dalam hitungan milidetik. Sementara di sisi **frontend**, kami belajar untuk membangun sistem UI/UX yang interaktif.

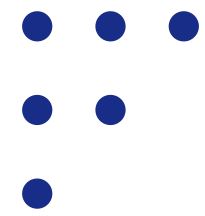


TESTING RESULT

selengkapnya ada di file docs/testing-result.md

1. Latency Test Result

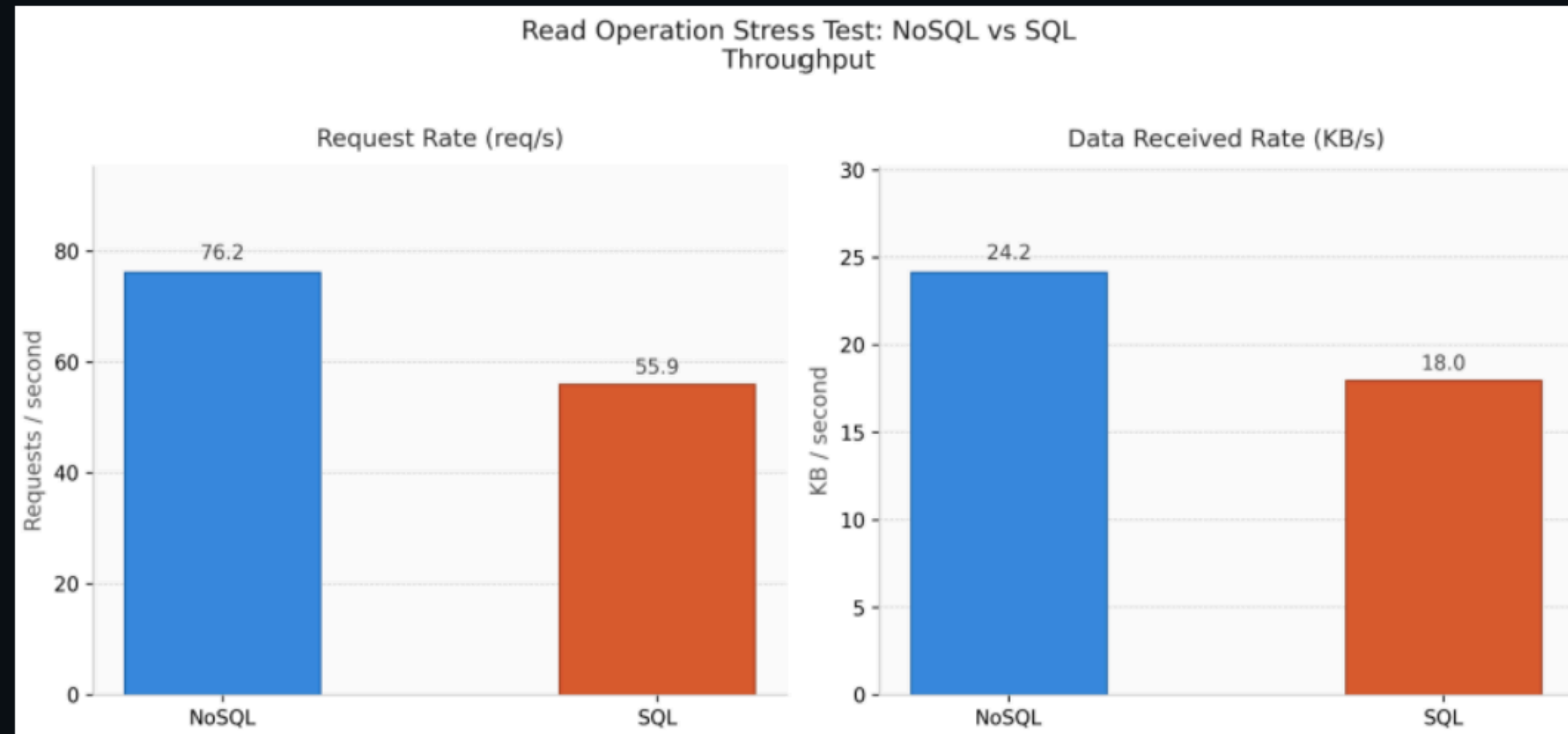


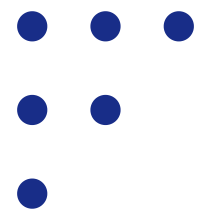


TESTING RESULT

selengkapnya ada di file docs/testing-result.md

2. Throughput Test Result





TESTING RESULT

selengkapnya ada di file docs/testing-result.md

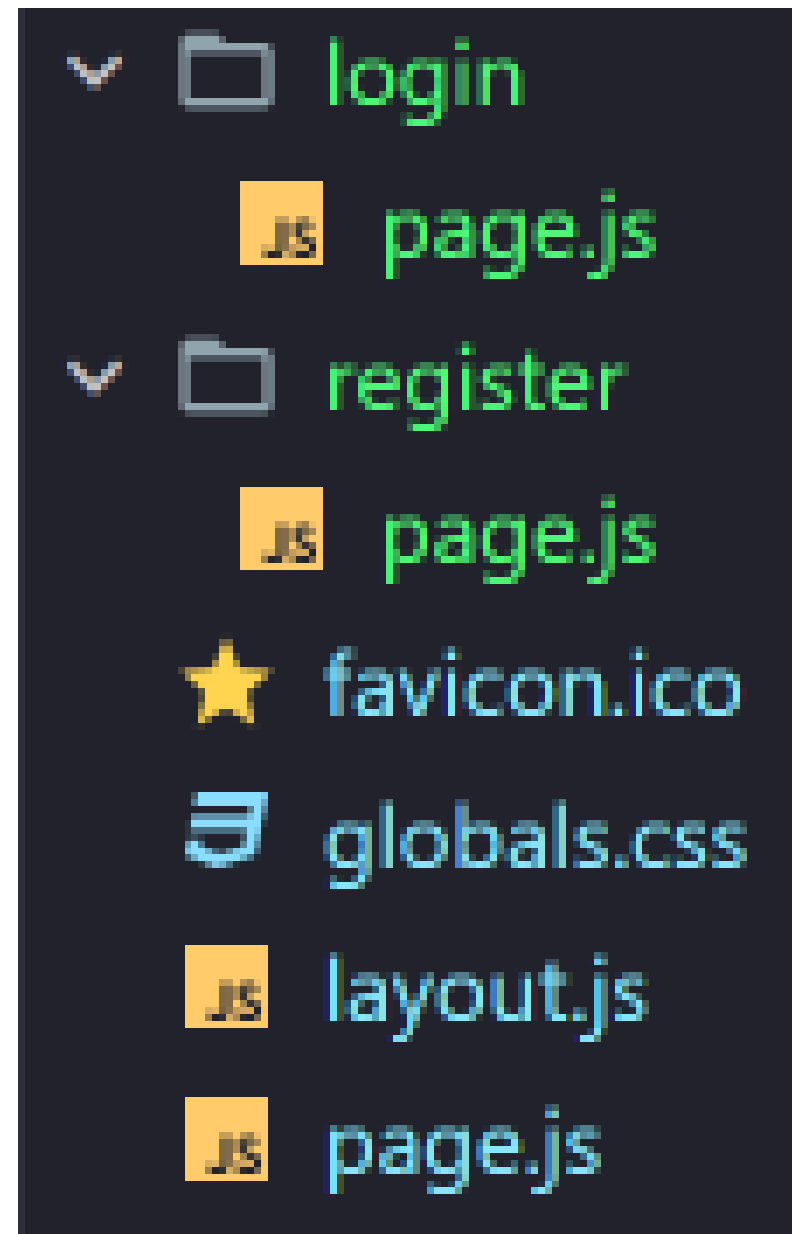
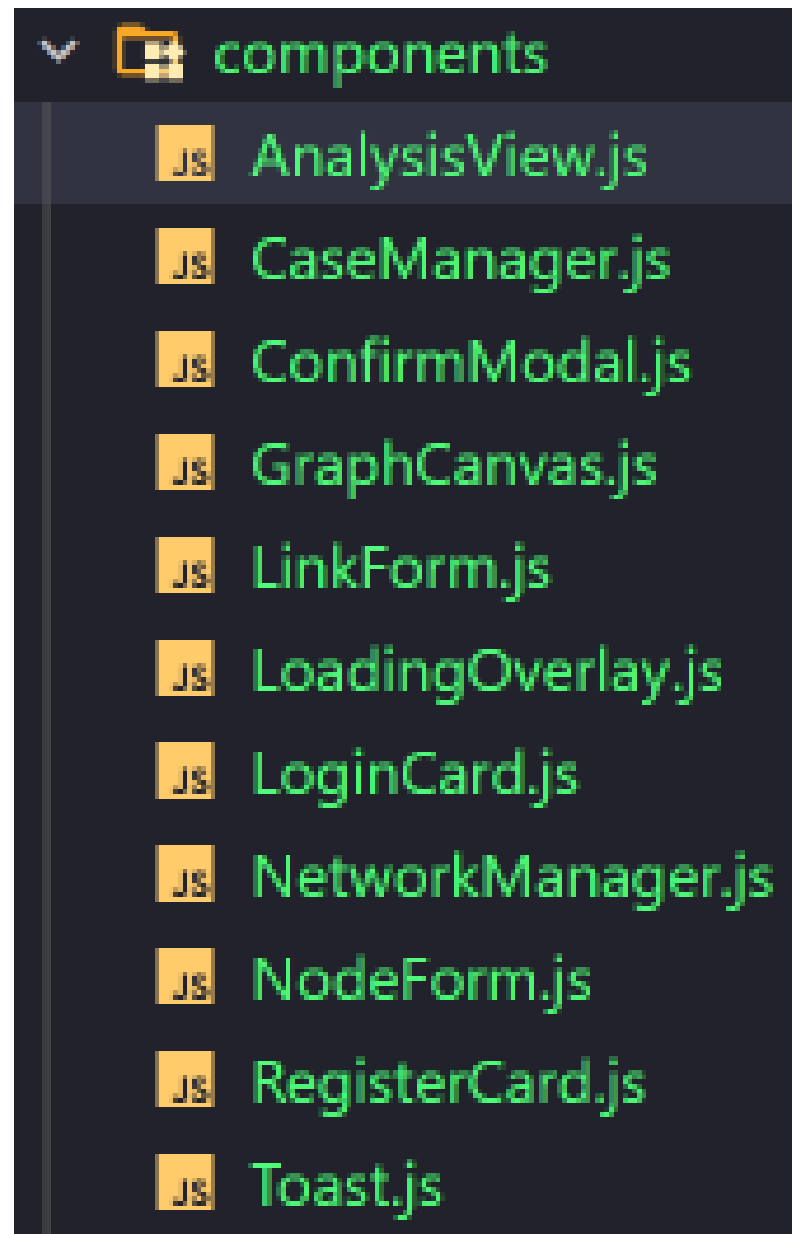
Analisis Hasil Pengujian Berdasarkan Grafik

Berdasarkan visualisasi grafik batang yang dihasilkan dari pengujian *stress test*, efisiensi performa sistem menunjukkan perbedaan yang sangat signifikan antara kedua arsitektur database:

1. **Evaluasi Matriks Latency (Waktu Respon):** Melalui grafik *Avg and Max Latency*, terbukti bahwa arsitektur NoSQL Graph memiliki performa komputasi yang jauh lebih ringan. Rata-rata waktu respon (*Average Latency*) untuk **NoSQL** hanya **154 ms**, berbanding terbalik dengan **SQL** yang meningkat hingga **245 ms**. Hal yang sama terlihat pada *Maximum Latency*, di mana batas atas lonjakan waktu respon NoSQL mampu diredam pada **387 ms**, sedangkan SQL melonjak hingga **523 ms** akibat tingginya beban kalkulasi pencarian indeks.
2. **Evaluasi Matriks Throughput (Produktivitas Data):** Dampak efisiensi latensi tersebut berbanding lurus dengan produktivitas server yang ditunjukkan pada grafik *Throughput*. Pada grafik *Request Rate*, **NoSQL** mampu melayani kecepatan transfer hingga **76.2 req/s** (requests per second), sedangkan **SQL** lebih lambat dengan kecepatan transfer **55.9 req/s**. Unggulnya produktivitas NoSQL ini juga dibuktikan oleh grafik *Data Received Rate*, di mana **NoSQL** mencatat kecepatan penyerapan dan transmisi data yang lebih tinggi sebesar **24.2 KB/s** dibandingkan **SQL** yang hanya mampu menyalurkan data sebesar **18.0 KB/s** dalam kondisi beban yang sama.



INDIVIDUAL CONTRIBUTIONS / MALIK (FRONTEND)



- **Folder Components:** membuat seluruh file komponen UI seperti GraphCanvas.js (visualisasi D3/Force Graph), AnalysisView.js, CaseManager.js, ConfirmModal.js, LinkForm.js, NodeForm.js, NetworkManager.js, LoadingOverlay.js, Toast.js, LoginCard.js, dan RegisterCard.js.
- **Halaman Frontend (/app/):** Mengerjakan struktur tampilan utama dan routing UI pada app/page.js, app/layout.js, app/globals.css, serta halaman autentikasi app/login/page.js dan app/register/page.js.

INDIVIDUAL CONTRIBUTIONS / NAUFAL (BACKEND)



- **Folder API Routes (/app/api/):** menyusun endpoint komunikasi pada `app/api/cases/route.js`, `app/api/cases/[id]/route.js` (untuk operasi CRUD kasus).
- **API Node & Relasi:** membuat handler untuk input entitas melalui `app/api/nodes/route.js` dan `app/api/links/route.js`.
- **API Autentikasi:** mengamankan jalur akses pada `app/api/auth/login/route.js` dan `app/api/auth/register/route.js`.

INDIVIDUAL CONTRIBUTIONS / EUGENIA (DATABASE)



```
import neo4j from 'neo4j-driver';

const uri = process.env.NEO4J_URI;
const user = process.env.NEO4J_USERNAME;
const password = process.env.NEO4J_PASSWORD;

let driver;

if (!global.neo4jDriver) {
  global.neo4jDriver = neo4j.driver(uri, neo4j.auth.basic(user, password));
}
driver = global.neo4jDriver;

export default driver;
```

A file explorer interface showing a file named '.env.local'.

- **Konfigurasi Koneksi (/lib/)**: menulis kode pada file lib/neo4j.js yang berisi instansiasi driver Neo4j dan manajemen session.
- **Environment Variables**: mengelola kredensial rahasia (URI, Username, Password) di dalam file .env.local

INDIVIDUAL CONTRIBUTIONS / HAIKAL (DATA MODELER & QUERY SPECIALIST)

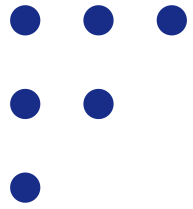


- **Injeksi query analisis:** berkolaborasi di dalam folder backend dengan menyusun raw Cypher strings (query algoritma fraud) yang dimasukkan ke dalam file `app/api/analysis/route.js` (misalnya query untuk mendeteksi circular transfer atau duplikasi NIK).
- **Injeksi query visualisasi:** menyusun query ekstraksi graph untuk kanvas visual yang dimasukkan ke dalam `app/api/graph/route.js`.

INDIVIDUAL CONTRIBUTIONS / NADIA (TECHNICAL WRITER & QA)



- Menulis file README.md di GitHub
- Melakukan *End-to-End Testing* dari form registrasi hingga memastikan datanya benar-benar masuk ke Neo4j Aura.
- Menyusun slide PPT.



TERIMA KASIH

Link github: https://github.com/naufalthecodemaker/Kelompok-6_FraudLens-Engine

